

ĐỀ KIỂM TRA CUỐI KHÓA

CODEBASE - PYTHON 2026

Đề số 01

Câu 1: Tính toán hóa đơn tiền điện

1.1. Mô tả bài toán

Thư đang train một model rất lớn nên phải giữ máy bật liên tục. Vì phải bật máy qua ngày đêm nên gia đình Thư tốn rất nhiều tiền điện. Bố mẹ Thư thấy thế nên không chu cấp tiền điện cho Thư nữa, mà máy tắt thì hỏng hết model của Thư mất!

May sao bạn là một đại gia sẵn sàng bao nuôi Thư để Thư tiếp tục được train model (Thư thật có phúc khi có người em như bạn!). Hãy tính tiền điện bạn sẽ chi cho Thư hàng tháng dựa trên số kWh đã tiêu thụ. Đơn giá được tính lũy tiến theo các mức sau:

- **Mức 1:** Với 50 kWh đầu tiên, giá là 1.678 VNĐ/kWh.
- **Mức 2:** Với các kWh từ 51 đến 100, giá là 1.734 VNĐ/kWh.
- **Mức 3:** Từ kWh thứ 101 trở đi, giá là 2.014 VNĐ/kWh.

1.2. Yêu cầu

Viết hàm `tinh_tien_dien(kwh)` thực hiện:

1. Nhận vào tham số `kwh` (kiểu số thực).
2. Sử dụng cấu trúc `if...elif...else` để tính tổng số tiền trước thuế.
3. Cộng thêm 10% thuế giá trị gia tăng (VAT) vào tổng số tiền.
4. Trả về kết quả cuối cùng là số thực, làm tròn đúng **4 chữ số thập phân**.

1.3. Khung mã nguồn

```
def tinh_tien_dien(kwh):  
    # Buoc 1: Kiem tra dieu kien kwh <= 0  
    if kwh <= 0:  
        return 0.0000  
  
    # Buoc 2: Tinh toan theo tung muc (if...elif...else)  
    # Viet code o day...  
  
    return result
```

1.4. Test Case mẫu

Input	Output
120	231968.0000

Giải thích:

$50 \times 1678 + 50 \times 1734 + 20 \times 2014 = 210,880 \text{ VNĐ.}$

Tổng sau thuế (10%): $210,880 \times 1.1 = 231,968 \text{ VNĐ.}$

Câu 2: Danh sách điểm thi hợp lệ

2.1. Yêu cầu bài toán

Hoàng là một tay săn học bổng khét tiếng. Để có được thành tích này Hoàng phải lên thu thập danh sách điểm số của các bạn trong lớp để cạnh tranh. Nhưng do lỗi nhập liệu, danh sách này chứa cả các giá trị không phải là số (ví dụ: chuỗi ký tự, giá trị rỗng).

Hãy giúp Hoàng viết hàm `loc_diem_hop_le(danh_sach_diem)` thực hiện các công việc sau:

- Duyệt qua từng phần tử trong danh sách đầu vào.
- Sử dụng khối `try...except` để cố gắng chuyển đổi từng phần tử sang kiểu số thực (float).
- Nếu chuyển đổi thành công và điểm số nằm trong khoảng từ 0 đến 10, hãy thêm điểm đó vào một danh sách mới.
- Nếu phần tử không thể chuyển đổi thành số (gây ra lỗi `ValueError` hoặc `TypeError`), hãy bỏ qua phần tử đó thay vì để chương trình dừng lại.

2.2. Kết quả trả về

Hàm cần trả về một **Dictionary** chứa hai thông tin:

1. `"diem_hop_le"`: Danh sách các điểm đã được lọc.
2. `"diem_trung_binh"`: Điểm trung bình của các điểm hợp lệ (làm tròn **4 chữ số thập phân**). Nếu không có điểm hợp lệ nào, giá trị này bằng 0.

2.3. Khung mã nguồn gợi ý

```
def loc_diem_hop_le(danh_sach_diem : list) -> dict:
    ket_qua = []

    for item in danh_sach_diem:
        try:
            # Co gang chuyen doi sang float
            # Viet code o day...
```

```

        # Kiem tra dieu kien diem tu 0 den 10
        # Viet code o day...

    # Neu loi thi bo qua (pass)
    except ValueError:
        pass
    except TypeError:
        pass

# Tinh diem trung binh va tra ve dictionary
# Viet code o day...
return {...}

```

2.4. Test Case mẫu

- Input:

```
["8.5", "abc", "10", "-2", "7.25", None]
```

- Output:

```
{
    "diem_hop_le": [8.5, 10.0, 7.25],
    "diem_trung_binh": 8.5833
}
```

- Xử lý:

- "8.5" → Hợp lệ.
- "abc" → Lỗi (ValueError), bỏ qua.
- "10" → Hợp lệ.
- "-2" → Ngoài khoảng 0-10, bỏ qua.
- "7.25" → Hợp lệ.
- None → Lỗi (TypeError), bỏ qua.

Câu 3: Chuẩn bị danh sách hậu cần

3.1. Yêu cầu bài toán

Để chuẩn bị cho talkshow diễn ra thành công mỹ mãn, ban Học thuật đóng vai trò là những nhân sự hậu cần, tất bật chuẩn bị yếu phẩm chạy talkshow. Các bạn ban Học thuật đã chuẩn bị đầy đủ danh sách vật dụng chính là file "c3_input.csv". Hãy giúp ban Học thuật tính toán chi phí dự trù từ file này.

3.1.1 Yêu cầu xây dựng Lớp (Object Oriented Programming)

Xây dựng lớp `SanPham` để quản lý thông tin hàng hóa trong kho với các thành phần sau:

- **Thuộc tính (Attributes):** `ma_sp` (mã sản phẩm), `ten_sp` (tên sản phẩm), `gia_ban` (số thực), `so_luong` (số nguyên).
- **Phương thức khởi tạo (`__init__`):** Gán giá trị cho các thuộc tính khi tạo đối tượng.
- **Phương thức `tinh_gia_du_tru()`:** Trả về tổng giá dự trữ của sản phẩm (bằng `gia_ban` nhân với `so_luong`).

3.2.2 Yêu cầu xử lý Tập tin (File Handling)

Viết hàm `doc_danh_sach_tu_file(ten_file_nhap)` thực hiện:

1. Mở tập tin có tên `ten_file_nhap` ở chế độ đọc ('r') với bảng mã `utf-8`.
2. Giả sử mỗi dòng dữ liệu trong file là thông tin sản phẩm theo cấu trúc `MaSP,TenSP,GiaBan,SoLuong`.
Nhiệm vụ của bạn là chuyển đổi mỗi dòng dữ liệu này thành từng đối tượng `SanPham`. Sau đó gộp tất cả các đối tượng này và trả về một danh sách (`list`).
3. **Xử lý ngoại lệ:** Sử dụng khối `try...except` để bắt lỗi `IOError` (lỗi nhập xuất file) và in thông báo lỗi nếu không thể ghi được file.

Viết hàm `ghi_bao_cao(danh_sach_sp, ten_file_xuat)` thực hiện:

1. Nhận vào một danh sách các đối tượng thuộc lớp `SanPham`.
2. Mở tập tin có tên `ten_file_xuat` ở chế độ ghi ('w') với bảng mã `utf-8`.
3. Với mỗi sản phẩm trong danh sách, ghi một dòng vào file theo định dạng:
`MaSP - TenSP - GiaTriTonKho`
4. **Xử lý ngoại lệ:** Sử dụng khối `try...except` để bắt lỗi `IOError` (lỗi nhập xuất file) và in thông báo lỗi nếu không thể ghi được file.

3.3. Khung mã nguồn gợi ý

```
class SanPham:
    def __init__(self, ma, ten, gia, sl):
        self.ma_sp = ma
        self.ten_sp = ten
        self.gia_ban = gia
        self.so_luong = sl

    def tinh_gia_du_tru(self):
        # Tính giá trị tồn kho của sản phẩm
        # Viết code ở đây...
        return ...
```

```

def doc_danh_sach_tu_file(ten_file_nhap : str) -> list:
    ds_doi_tuong = []
    try:
        # Doc file input va xu ly tung dong
        # Viet code o day...
        return ds_doi_tuong
    except FileNotFoundError:
        print(f"Loi: Khong tim thay file {ten_file_nhap}")
        return []

def ghi_bao_cao(danh_sach_sp : list, ten_file_xuat : str):
    try:
        # Xu ly san pham va ghi ra ten_file_xuat
        # Viet code o day...
        print(f"Da ghi bao cao vao file {ten_file_xuat}")
    except IOError:
        print("Loi: Khong the ghi du lieu vao file!")

# (OPTIONAL) CHUONG TRINH CHINH DE TEST
if __name__ == "__main__":
    file_vao = "c3_input.csv"
    file_ra = "c3_output.txt"

    # Buoc 1: Doc tu file de tao danh sach doi tuong
    danh_sach = doc_danh_sach_tu_file(file_vao)

    # Buoc 2: Neu co du lieu thi ghi bao cao
    if danh_sach:
        ghi_bao_cao(danh_sach, file_ra)

```

3.4. Test case mẫu

Input: file "c3_input.csv"

```

SP01 , Hoa , 150 , 6
SP02 , Qua , 20.5 , 15

```

Output: file "c3_output.txt"

```

SP01 - Hoa - 900.0000
SP02 - Qua - 307.5000

```

Câu 4: Xử lý dữ liệu cơ bản với NumPy và Pandas

4.1. Yêu cầu bài toán

Cửa hàng bán hoa cho bạn Học thuật lưu trữ dữ liệu doanh thu bán hàng trong nhiều ngày dưới dạng danh sách số liệu. Để thuyết phục bác bán hoa mặc cả, bạn thể hiện bằng

cách phân tích dữ liệu doanh thu cho bác. Nếu làm tốt bác sẽ giảm giá cho bạn. Bạn cần sử dụng thư viện **NumPy** và **Pandas** để phân tích dữ liệu cơ bản.

Cho dữ liệu đầu vào gồm:

- `doanh_thu`: Danh sách doanh thu từng ngày (đơn vị: triệu VNĐ).
- `ngay`: Danh sách ngày tương ứng.

Ví dụ:

```
ngay = ["2024-01-01", "2024-01-02",  
        "2024-01-03", "2024-01-04"]  
  
doanh_thu = [12.5, 15.0, 9.5, 20.0]
```

4.2. Yêu cầu thực hiện

Viết hàm `phan_tich_doanh_thu(ngay, doanh_thu)` thực hiện các công việc sau:

1. Sử dụng **NumPy** để:

- Chuyển danh sách doanh thu (`list`) thành mảng NumPy (`ndarray`).
- Tính:
 - Tổng doanh thu.
 - Doanh thu trung bình.
 - Doanh thu lớn nhất.
 - Doanh thu nhỏ nhất.

2. Sử dụng **Pandas** để:

- Tạo một `DataFrame` gồm hai cột:
 - "Ngày"
 - "DoanhThu"
- Thêm một cột mới "DanhGia":
 - "Tot" nếu doanh thu ≥ 15
 - "TrungBinh" nếu doanh thu từ 10 đến dưới 15
 - "Thap" nếu doanh thu < 10
 - *Gợi ý*: Thiết kế hàm `danh_gia` rồi dùng `.apply(danh_gia)` cho cột mới

3. Trả về một Dictionary chứa:

- "tong_doanh_thu"
- "trung_binh"
- "cao_nhat"
- "thap_nhat"
- "bang_du_lieu" (`DataFrame` kết quả)

4.3. Khung mã nguồn gợi ý

```
import numpy as np
import pandas as pd

def phan_tich_doanh_thu(ngay, doanh_thu):

    # Buoc 1: Chuyen doanh_thu thanh mang NumPy
    arr = np.array(doanh_thu)

    # Buoc 2: Tinh cac thong ke co ban
    # Viet code o day ...

    # Buoc 3: Tao DataFrame bang Pandas
    df = pd.DataFrame({
        "Ngay": ngay,
        "DoanhThu": doanh_thu
    })

    # Buoc 4: Them cot DanhGia
    # Viet code o day ...

    # Buoc 5: Tra ve ket qua
    return {
        "tong_doanh_thu": round(tong, 4),
        "trung_binh": round(trung_binh, 4),
        "cao_nhat": round(cao_nhat, 4),
        "thap_nhat": round(thap_nhat, 4),
        "bang_du_lieu": df
    }

# (Optional) Chương trình thực thi chạy thử
if __name__ == "__main__":
    ngay = ["2024-01-01", "2024-01-02", "2024-01-03", "2024-01-04"]
    doanh_thu = [12.5, 15.0, 9.5, 20.0]

    ket_qua = phan_tich_doanh_thu(ngay, doanh_thu)

    # In ket qua thong ke
    print("Tong doanh thu:", ket_qua["tong_doanh_thu"])
    print("Trung b  =inh:", ket_qua["trung_binh"])
    print("Cao nhat:", ket_qua["cao_nhat"])
    print("Thap nhat:", ket_qua["thap_nhat"])

    # In DataFrame
    print("\nBang du lieu:")
    print(ket_qua["bang_du_lieu"])
```

4.4. Test Case mẫu

Input

```
["2024-01-01", "2024-01-02", "2024-01-03", "2024-01-04"]  
[12.5, 15.0, 9.5, 20.0]
```

Output

```
{  
  "tong_doanh_thu": 57.0,  
  "trung_binh": 14.25,  
  "cao_nhat": 20.0,  
  "thap_nhat": 9.5,  
  "bang_du_lieu":  
}
```

DataFrame kết quả

Ngày	DoanhThu	DanhGia
2024-01-01	12.5	TrungBinh
2024-01-02	15.0	Tot
2024-01-03	9.5	Thap
2024-01-04	20.0	Tot